# Demonstração — DynamoDB na AWS

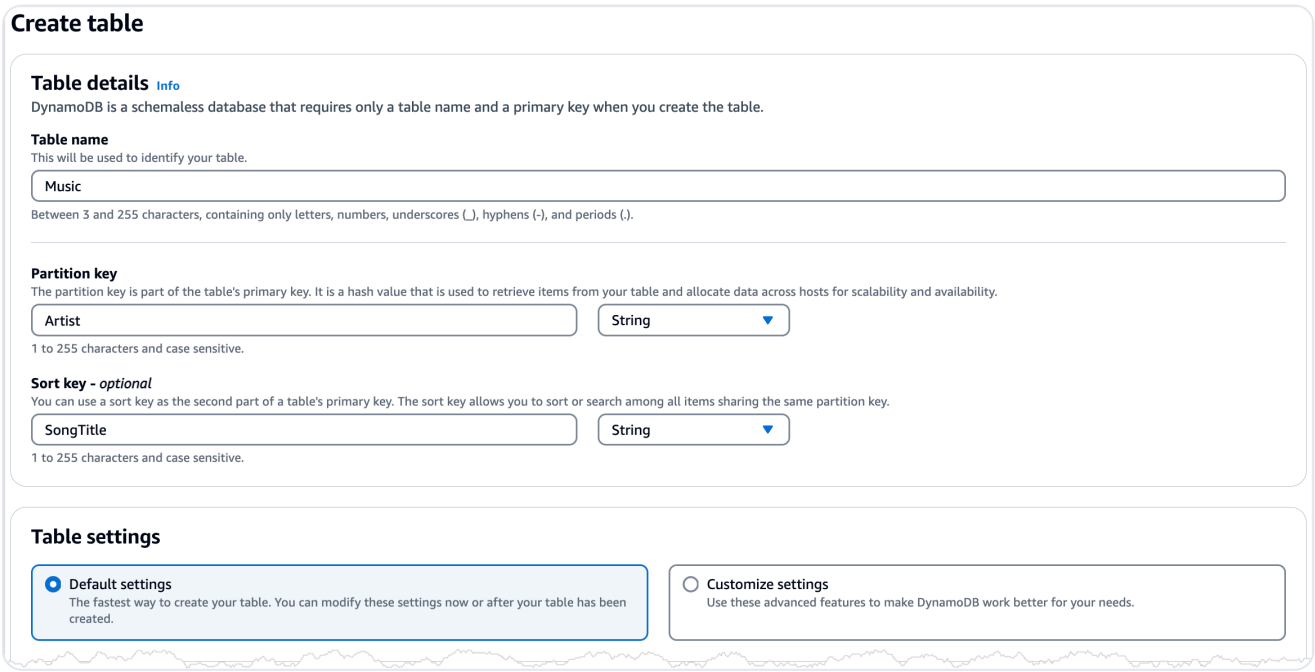
Partes A, B e C · Vítor Raimundo Fernandes Gabínio · Computação de Alto Desempenho · UEPB

## 3 Parte A — Console: tabela, itens e consultas

Baseado no tutorial oficial Getting started with DynamoDB.

### 3.1 Criar a tabela `Music`

1. Na barra de busca do console, digite **DynamoDB** e abra o serviço.
2. No menu da esquerda, **Tables** → **Create table**.
3. **Table name:** `Music` · **Partition key:** `Artist` (String) · **Sort key:** `SongTitle` (String).
4. Em **Table settings**, deixe **Default settings**. Confira no resumo que o **Capacity mode** está como **On-demand** (se não estiver, escolha **Customize settings** e selecione On-demand).
5. **Create table**. Em alguns segundos o status muda para **Active**.



Tela de criação da tabela (imagem da documentação da AWS).

### 3.2 Inserir itens

1. Clique na tabela **Music** → **Explore table items** → **Create item**.
2. Preencha `Artist` e `SongTitle`. Para adicionar outros campos: **Add new attribute** → escolha o tipo.
3. Crie estes 4 itens (note que eles **não têm os mesmos atributos** — é de propósito, para mostrar que não há esquema fixo):

| Artist | SongTitle | Outros atributos |
|---|---|---|
| No One You Know | Call Me Today | AlbumTitle (String) = Somewhat Famous · Awards (Number) = 1 |
| Acme Band | Happy Day | AlbumTitle = Songs About Life · Awards = 10 |
| Acme Band | PartiQL Rocks | AlbumTitle = Another Album Title · Awards = 8 |
| Acme Band | Still in Love | Genre (String) = Rock · Year (Number) = 2015 |

**Atalho:** em **Create item** existe a opção **JSON view**; dá para colar o item pronto. Exemplo (desligue a opção "View DynamoDB JSON" para usar JSON simples):

```
{
  "Artist": "Acme Band",
  "SongTitle": "Still in Love",
  "Genre": "Rock",
  "Year": 2015
}
```

### 3.3 Query pela chave

1. Ainda em **Explore table items**, abra **Scan or query items** e selecione **Query**.

2. **Partition key:** `Acme Band` → **Run**. Devolve só as 3 músicas da banda, já ordenadas pela sort key.
3. (Opcional) Adicione condição na sort key: **Begins with** `H` → devolve só "Happy Day".

### 3.4 PartiQL (sintaxe parecida com SQL)

1. No menu da esquerda, **PartiQL editor**.
2. Cole e clique em **Run**:

```
SELECT * FROM "Music" WHERE Artist = 'Acme Band'
```

```
INSERT INTO "Music" VALUE {'Artist': 'UEPB Band', 'SongTitle': 'Alto Desempenho', 'Year': 2026}
```

---

## 4 Parte B — CloudShell: o teste de desempenho

O script `demo/demo_dynamodb.py` (está na pasta do projeto) faz quatro coisas:

1. cria a tabela `LeiturasSensores` (partition key `sensor_id`, sort key `ts`, on-demand);
2. grava 2.000 leituras de 20 sensores — 200 **uma por vez** e 1.800 com **16 threads em paralelo** (lotes de 25 com `BatchWriteItem`) — e compara itens/segundo;
3. faz 300 leituras por chave (`GetItem`) e mostra a latência média, p50, p90 e p99, com leitura **eventual** e **forte**;
4. busca as leituras de um sensor com **Query** e com **Scan** e compara itens lidos, capacidade consumida e tempo.

### 4.1 Abrir o CloudShell e enviar o script

1. No console (na **mesma região** da tabela), clique no ícone de terminal `>_` na barra superior, ou pesquise **CloudShell**. Espere o terminal abrir (~30 s na primeira vez).
2. Menu **Actions** → **Upload file** → selecione `demo_dynamodb.py` (pasta `demo` do projeto).
3. Confira: `ls` deve listar o arquivo.

▼ **Sem upload? Cole o script direto no terminal**

Digite `cat > demo_dynamodb.py << 'EOF'`, tecle Enter, cole todo o conteúdo do arquivo, tecle Enter, digite `EOF` e Enter. O CloudShell pode mostrar um aviso de “Safe Paste” — confirme com **Paste**.

### 4.2 Rodar

```
python3 demo_dynamodb.py
```

Leva cerca de 20–40 segundos (a maior parte é a criação da tabela). Saída esperada (os números vão variar):

```
================================================================
  2) Escrita de 2000 itens: sequencial x paralela
================================================================
Sequencial (PutItem, 1 por vez)     :   200 itens em  1.2 s →    170 itens/s
Paralela   (16 threads, lotes de 25):  1800 itens em  0.6 s →   3000 itens/s
Ganho com paralelismo: 17.6x

================================================================
  3) Latência de 300 leituras por chave (GetItem)
================================================================
Leitura eventual     média   4.10 ms | p50   3.90 | p90   5.00 | p99   9.80 ms
Leitura forte        média   4.40 ms | p50   4.20 | p90   5.30 | p99  10.50 ms

================================================================
  4) Buscar as leituras de UM sensor: Query x Scan
================================================================
QUERY  → retornou  100 | leu   100 itens |    1.0 RCU |     8.0 ms
SCAN   → retornou  100 | leu  2000 itens |   20.0 RCU |   120.0 ms
```

Valores ilustrativos — use os da sua execução na narração.

### 4.3 Como interpretar (para a narração)

- **Sequencial × paralela:** uma requisição por vez desperdiça tempo esperando a rede. Com várias threads e lotes, muitas requisições ficam em voo ao mesmo tempo e o DynamoDB as atende em paralelo, em partições diferentes. É a mesma ideia de paralelismo da disciplina.
- **Latência:** o tempo medido inclui a ida e volta pela rede dentro da região. O importante é o **p99 continuar baixo**: "latência previsível".
- **Eventual × forte:** a diferença costuma ser pequena; a leitura forte custa o dobro de capacidade porque precisa do líder.
- **Query × Scan:** a Query vai direto na partição da chave; o Scan lê a tabela inteira e só depois filtra. Mesmo resultado, ~20× mais capacidade consumida. Em uma tabela de terabytes a diferença seria gigantesca.

> **Rode duas ou três vezes** antes de gravar. Na segunda execução a tabela já existe e o script a reaproveita (os itens são sobrescritos, não duplicados). Se quiser filmar a criação da tabela, apague antes com `python3 demo_dynamodb.py --apagar`.

---

## 5 Parte C — Métricas

1. Volte ao DynamoDB → `Tables` → **LeiturasSensores** → aba `Monitor`.
2. Mostre os gráficos de capacidade de leitura/escrita consumida e de latência. Os pontos aparecem com 1 a 5 minutos de atraso — então grave esta parte **alguns minutos depois** de rodar o script.

### O que isso quer dizer?

**Parte A:** criamos uma tabela sem configurar nenhum servidor e encontramos as músicas só informando a chave. **Isso quer dizer que** o DynamoDB cuida da infraestrutura sozinho, e a chave é o caminho direto até o dado.

**Escrita:** gravar em paralelo foi muitas vezes mais rápido do que gravar um item por vez. **Isso quer dizer que** os dados estão espalhados em várias partições que trabalham ao mesmo tempo — é a escala horizontal acontecendo na prática.

**Latência:** as leituras responderam em poucos milissegundos, e até as mais lentas (p99) continuaram rápidas. **Isso quer dizer que** o tempo de resposta é previsível, e não apenas rápido na média.

**Query × Scan:** a Query leu 100 itens; o Scan leu 2.000 para chegar ao mesmo resultado. **Isso quer dizer que**, no DynamoDB, o desempenho depende de buscar pela chave — numa tabela gigante, essa diferença seria enorme.

**Parte C:** as métricas mostraram o uso gerado pelo teste, sem instalarmos nada. **Isso quer dizer que** o serviço já vem monitorado e pronto para produção.

---

**Em resumo:** o DynamoDB é rápido em qualquer escala porque divide os dados pela chave e trabalha em paralelo — o mesmo princípio da computação de alto desempenho que estudamos na disciplina.